

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

ANÁLISIS ARQUITECTÓNICO - Dashboard 'The Power Grid'

PATRÓN ARQUITECTÓNICO:

Se adopta una arquitectura Frontend SPA (Single Page Application) puramente cliente, sin dependencias de frameworks externos. El patrón central es MVC adaptado a Vanilla JS: el estado global actúa como Modelo, los módulos de render como Vista, y los event handlers como Controlador. Se aplica el patrón Observer/Pub-Sub para que los cambios de estado propaguen actualizaciones a múltiples subsistemas (KPIs, Canvas, CSS) de forma desacoplada.

CAPAS DE LA APLICACIÓN:

1. Capa de Estado (State Management): Un objeto global 'AppState' inmutable por referencia directa, modificado únicamente a través de funciones puras de transición. Contiene: estado booleano de 5 módulos, flag overdrive, valores actuales de KPIs y configuración KPI_CONFIG con boosts por herramienta.
2. Capa de Render Canvas: Dos canvas superpuestos con z-index diferenciado. 'particleCanvas' gestiona el sistema de partículas con pool de objetos para evitar GC pressure. 'connectionCanvas' dibuja bezier curves entre nodos y el núcleo, con pulsos de datos animados mediante interpolación paramétrica.
3. Capa de UI DOM: Componentes de KPI (velocímetro, radar, barras de progreso) renderizados en canvas dedicados por widget. Animaciones de números via requestAnimationFrame con easing easeOutCubic.
4. Capa de Estilos Dinámicos: CSS Custom Properties modificadas via JS (document.documentElement.style.setProperty) para transiciones de tema global en modo Overdrive.

SEGURIDAD Y BUENAS PRÁCTICAS:

- Sin dependencias externas: superficie de ataque mínima, sin vulnerabilidades de supply chain.
- Sin llamadas a APIs externas: no hay riesgo de CORS, SSRF ni exposición de datos.
- Sanitización de DOM: uso exclusivo de textContent para inyección de valores numéricos, nunca innerHTML con datos dinámicos.
- CSP-friendly: sin eval(), sin inline handlers en HTML, eventos registrados via addEventListener.
- Performance: Object pooling para partículas, cancelación de animationFrame al salir del viewport (IntersectionObserver), throttling de resize events.

ESTRUCTURA DE MÓDULOS JS:

- main.js: Bootstrap, inicialización y orquestación.
- state/appState.js: Estado global y funciones de transición puras.
- config/kpiConfig.js: Configuración de KPIs, boosts por herramienta, referencias DOM.
- config/moduleConfig.js: Configuración visual de los 5 nodos (colores, posiciones, labels).
- canvas/particleSystem.js: Sistema de partículas con pool, física simple.
- canvas/connectionRenderer.js: Bezier curves, pulsos de datos, glow effects.
- canvas/coreRenderer.js: Núcleo Jira, anillos orbitales, animación de rotación.
- ui/kpiPanel.js: Actualización de paneles laterales, animación de números.
- ui/speedometer.js: Widget velocímetro en canvas dedicado.
- ui/radarChart.js: Widget radar chart 7 ejes en canvas dedicado.
- ui/nodeRing.js: Renderizado y posicionamiento radial de los 5 nodos.
- ui/overdriveOverlay.js: Pantalla superpuesta modo Overdrive, efectos neón.
- ui/doraBar.js: Barra inferior con métricas DORA y trazabilidad.
- utils/animation.js: Funciones de easing, interpolación, animateValue.
- utils/mathHelpers.js: Cálculos geométricos para posicionamiento radial y canvas.
- styles/main.css: Variables CSS, layout base, Flexbox grid.
- styles/modules.css: Estilos específicos de nodos, colores por herramienta.
- styles/animations.css: @keyframes para rotación, latidos, parpadeo Overdrive.
- styles/overdrive.css: Tema neón cyan para modo Overdrive.
- styles/kpi-panels.css: Estilos de paneles laterales y barra inferior.

DECISIONES DE DISEÑO CLAVE:

- requestAnimationFrame unificado: un único loop maestro en main.js llama a todos los subsistemas de render en orden, evitando múltiples loops compitiendo.
- Separación canvas/DOM: los KPI numéricos viven en DOM para accesibilidad, los efectos visuales en canvas para performance.
- Posicionamiento radial calculado matemáticamente: las posiciones de los 5 nodos se calculan con trigonometría (cos/sin) relativas al centro del viewport, responsive a resize.
- Modo Overdrive como estado derivado: se detecta automáticamente cuando todos los módulos están activos, sin lógica especial adicional.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: APROBADO | Iteraciones: 1
- src/main.js | Estado: APROBADO | Iteraciones: 2
- src/state/appState.js | Estado: APROBADO | Iteraciones: 1
- src/config/kpiConfig.js | Estado: APROBADO | Iteraciones: 1
- src/config/moduleConfig.js | Estado: APROBADO | Iteraciones: 1
- src/canvas/particleSystem.js | Estado: WARNING (Forzado) | Iteraciones: 3
- src/canvas/connectionRenderer.js | Estado: APROBADO | Iteraciones: 1
- src/canvas/coreRenderer.js | Estado: APROBADO | Iteraciones: 1
- src/ui/kpiPanel.js | Estado: APROBADO | Iteraciones: 1
- src/ui/speedometer.js | Estado: APROBADO | Iteraciones: 1
- src/ui/radarChart.js | Estado: APROBADO | Iteraciones: 1
- src/ui/nodeRing.js | Estado: APROBADO | Iteraciones: 1
- src/ui/overdriveOverlay.js | Estado: APROBADO | Iteraciones: 1
- src/ui/doraBar.js | Estado: APROBADO | Iteraciones: 1
- src/utils/animation.js | Estado: APROBADO | Iteraciones: 1
- src/utils/mathHelpers.js | Estado: APROBADO | Iteraciones: 1
- styles/main.css | Estado: APROBADO | Iteraciones: 1
- styles/modules.css | Estado: APROBADO | Iteraciones: 1
- styles/animations.css | Estado: APROBADO | Iteraciones: 1
- styles/overdrive.css | Estado: APROBADO | Iteraciones: 1
- styles/kpi-panels.css | Estado: APROBADO | Iteraciones: 1